

Operating Systems — 歷屆考古題解答（2011–2016）

題號對應 [考古題彙整.md](#)。標注 ★ 者為每年必考，★★ 為高頻考題。

壹、中斷與 I/O 機制

Q1 | Interrupt-based vs Busy-waiting ★

With **busy-waiting (polling)**, the CPU continuously loops checking whether the I/O device is ready, wasting a large amount of CPU time — CPU is idle during the entire wait.

With **interrupt-based I/O**, the CPU initiates I/O and continues executing other processes. When the device completes, it sends an **interrupt signal** to notify the CPU. The CPU is **always kept busy**, significantly increasing CPU utilization.

Q2 | Hardware Interrupt Handling ★

1. **Device controller** stores data in its local buffer. When I/O is complete, it raises a signal on the **interrupt-request line**.
 2. CPU checks the interrupt-request line after every instruction.
 3. CPU receives the interrupt and performs **state saving** — saves the current process state (PC, registers) into the PCB.
 4. CPU transfers to a generic interrupt-handling routine, then uses the interrupt number to index into the **interrupt vector** to find the address of the **interrupt-specific handler**.
 5. The **interrupt-specific handler** executes (e.g., copies data from the device controller's local buffer to main memory).
 6. CPU executes a return-from-interrupt instruction, **restores the saved state**, and resumes the interrupted process.
-

Q3 | Modern Interrupt Mechanism

（來源：2016midA 學生答案）

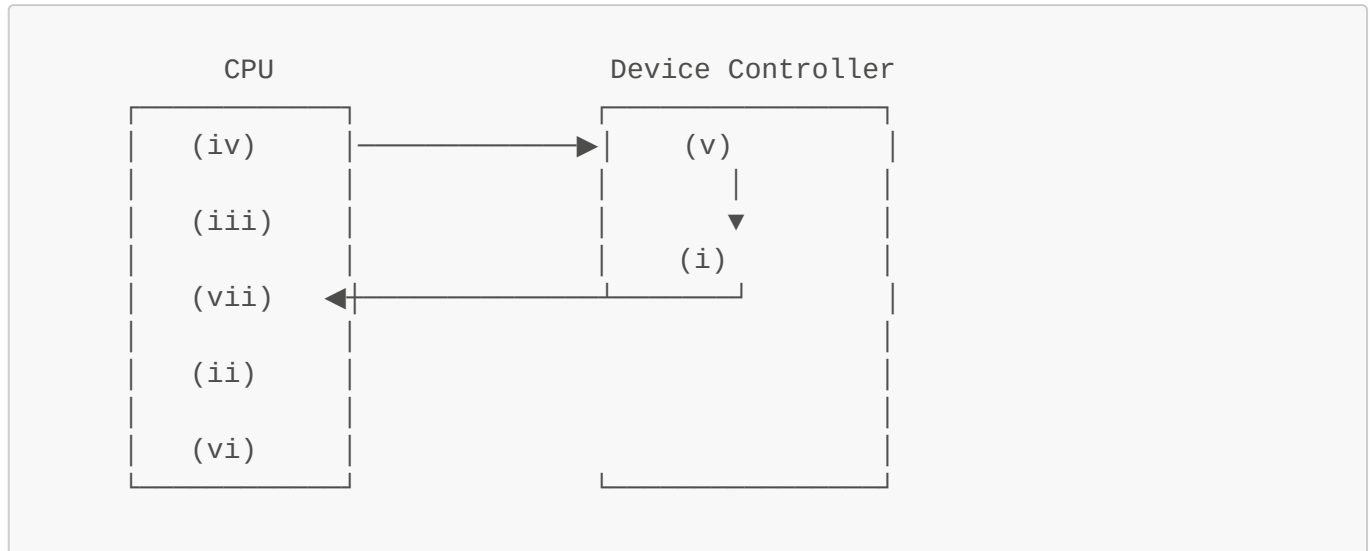
The modern interrupt mechanism accelerates interrupt operations by using the **interrupt vector**. The interrupt sent by I/O devices will contain an index of the interrupt, which represents what it wants to do. The interrupt handler checks the index in the interrupt vector, performing a quick response and returning to the interrupted process quickly.

It doesn't have to keep checking I/O status — some I/O devices will notify the interrupt handler via the **interrupt-request line** when ready.

Key points:

- Interrupt vector enables **direct lookup** by index instead of sequential search
- Device uses interrupt-request line → CPU only responds when needed
- No continuous polling → more efficient CPU usage

Q4 | Interrupt-driven I/O Cycle Flowchart ★



- **(iv)** device driver initiates I/O → sends command to device controller
- **(iii)** CPU executing, checks interrupt-request line between instructions
- **(v)** Controller initiates I/O (performs actual device operation)
- **(i)** Input ready / output complete / error → generates interrupt signal
- **(vii)** CPU receiving interrupt, transfer control to interrupt handler
- **(ii)** Interrupt handler processes data, return from interrupt
- **(vi)** CPU resumes processing of interrupted process

Q5 | Interrupt Vector 的好處

Using the interrupt vector, the CPU can use the interrupt number to **directly index into the table** to find the address of the corresponding interrupt-specific handler. This eliminates the need to **sequentially search through all handlers**, greatly speeding up interrupt processing and supporting many different types of interrupts.

Q6 | Device Controllers and Device Drivers

- **Device controller (硬體)** : A hardware component that sits between the device and the system bus. It reads or writes data stored in the device, has its own **local buffer**, and raises a signal on the interrupt-request line when I/O is complete.
- **Device driver (軟體)** : A software module in the OS kernel that tells the OS how to communicate with a specific device controller. It provides a **uniform interface** so the OS can interact with different devices in a standardized way. Device drivers send requests to device controllers; if the controller receives the request, it asks the device to do I/O operations.

Q7 | I/O Devices and CPU Execute Concurrently

(2012 Q1a)

When I/O devices and the CPU execute concurrently, it means that after the CPU issues an I/O request, the **device controller** takes over and performs the I/O operation independently. The CPU does **not** wait for the I/O to complete — it continues executing other processes or instructions. When the I/O is complete, the device controller notifies the CPU via an **interrupt**, allowing both to make progress simultaneously.

貳、系統結構 (System Structures)

Q8 | API Function Invoking a System Call ★

必答四要素：**software interrupt**、**dual-mode**、**system call number**、**table of code pointer**

1. The user program calls an API function (e.g., `read()`).
 2. The API function triggers a **software interrupt (trap)**, causing the CPU to switch from **user mode** to **kernel mode (dual-mode operation)**.
 3. The **system call number (index)** is placed into a designated register (e.g., EAX).
 4. The CPU uses this index to look up the **table of code pointer** (system call table) to find the memory address of the corresponding kernel function.
 5. The CPU jumps to that address and executes the kernel service.
 6. After the service completes, the CPU switches back to **user mode** and returns the result to the calling program.
-

Q9 | Why Prefer API over Direct System Calls ★

1. **Portability (可攜性)** : API functions provide a consistent interface across different operating systems. A program written using the POSIX API can run on any POSIX-compliant OS without modification. If invoking system calls directly, different OSes have different call numbers — the program cannot be ported.
 2. **Simplicity (簡便性)** : Directly invoking system calls is tedious and requires understanding low-level details (register conventions, call numbers, etc.). API functions provide a simpler, more intuitive interface.
-

Q10 | Passing MANY Parameters to OS

Use the **memory block (block method / parameters marshalling)**:

Store all parameters in a **block (table) in memory**, then place the **memory address** of that block into a register and pass it to the OS. The OS reads all parameters from that address. This is the method used by Linux and Solaris.

Advantage: Can pass **any number of parameters** without being limited by the number of registers.

Q11 | Layered OS Structure — Pros and Cons

Pros:

- **Easy to debug / decouple (Modularity):** Each layer can be verified independently, starting from Layer 0 (hardware). If a bug exists in a layer, all layers below are already verified, so the bug must be in that layer.

Cons:

- **Hard to define layers:** Difficult to decide which functions belong in which layer — dependencies may span layers.
- **Poor performance:** A service request must traverse multiple layers; each layer adds function call overhead, making it less efficient than a monolithic approach.

Q12 | Microkernel System Structure — Pros and Cons

Pros:

- **Reliable:** Non-essential services run as user-level servers. If one server crashes, it doesn't crash the kernel.
- **Portable:** The small kernel is easier to port to new hardware platforms.
- **Secure:** Fewer functions in kernel mode — easier to control access.

Cons:

- **Performance overhead:** Services require communication between user space and kernel space via **message passing**, causing frequent mode switches and significant **performance degradation**.

Q13 | Modular Kernel — Advantages

Over Layered: The modular kernel has higher efficiency since it **doesn't have to go through so many layers**. Any module can directly communicate with any other module without going through intermediate layers.

Over Microkernel: Modules run in **kernel space** — no need for message passing between user space and kernel space. This avoids the performance overhead of the microkernel while still maintaining most of the OS's correct execution.

Q14 | Dual-mode Operation

Definition: The OS supports two execution modes: **user mode (mode bit = 1)** and **kernel mode (mode bit = 0)**. Operations that could harm the system can only be executed in kernel mode. When an interrupt or trap occurs, the mode switches from user to kernel; when the OS completes, it switches back to user mode.

How it protects the OS: When a program in user mode attempts to execute a **privileged instruction**, the hardware treats it as illegal, triggers a trap, and hands control to the OS. This ensures that user programs

cannot directly access the kernel or critical system resources, preventing errant or malicious programs from damaging the system.

Q15 | Higher-level Language for OS Implementation

- 1. The code is easy for humans to **read and understand**.
- 2. Code can be **written faster**.
- 3. Programmers can share code with teammates, use **compilers and debuggers**, and develop more modules.
- 4. **Most importantly:** The OS is **easier to port** to other hardware platforms — only a small amount of machine-specific code needs to change.

Q16 | CLI — Two Implementation Approaches

	Approach 1: Built-in commands	Approach 2: System programs (UNIX style)
How	Command code is inside the interpreter	Interpreter searches PATH, finds and loads corresponding system program
Pros	Execution is fast	Interpreter is small in size; easy to add new commands
Cons	Interpreter is large ; rare commands waste memory; adding commands requires modifying the interpreter	Takes longer to execute — must search and load program each time

Most commands in shell are to implement file system. If a command interpreter contains all commands, the size of interpreter will be huge but quick. Some commands are rare to use and loaded in memory will be waste. With system programs, interpreter can be small in size but takes a longer time to execute a command.

Q17 | Multiprogramming vs Timesharing

- **Multiprogramming:** Keeps multiple jobs simultaneously in memory. When one job waits for I/O, the CPU switches to another job. Goal: **maximize CPU utilization**.
- **Time-sharing (Timesharing):** An **extension of multiprogramming**. The CPU switches between jobs so rapidly (using time quantum) that each user feels they have a dedicated computer. Goal: **minimize response time** and provide interactive use (typically response time < 1 second).

Key difference: Multiprogramming maximizes CPU utilization; time-sharing minimizes response time. **Time-sharing is an extension of multiprogramming.**

Q18 | Asymmetric vs Symmetric Multiprocessing

- **Asymmetric Multiprocessing (AMP):** One **master processor** controls the system and assigns tasks to **slave processors**. Only the master runs the OS; slaves execute user programs. Simpler design but

the master is a bottleneck.

- **Symmetric Multiprocessing (SMP):** All processors are **peers** — each runs its own copy of the OS and can access shared memory. Most modern systems use SMP. More complex to design but better performance and no single point of failure.

Q19 | How OS Supports New Devices

Operating systems support new devices through **device drivers**. Device drivers are **software** that serve as a medium between the device controller and the OS. They communicate with device controllers and provide a uniform interface for the OS to interact with different hardware. In this way, devices can be recognized by any operating system and be supported.

參、行程 (Processes)

Q20 | Four Components of a Process in Memory

Component	Description
Text (Code)	The executable program code
Data	Global and static variables
Heap	Memory dynamically allocated at runtime (via malloc/new)
Stack	Temporary storage for function parameters, return addresses, local variables

Q21 | Process State — Situation判斷★

版本 A（2011）答案

Situation	State
(a) is selected by a long-term scheduler	ready
(b) is selected by a short-term scheduler	running
(c) invokes a blocking system call	waiting
(d) completes a blocking system call	ready
(e) is interrupted by an interrupt	ready

版本 B（2012 / 2013）答案

Situation	State
(a) after it is selected by a long-term scheduler	ready

Situation	State
(b) before it is selected by a short-term scheduler	ready
(c) after it is selected by a short-term scheduler	running
(d) before it invokes a blocking system call	running
(e) after it invokes a blocking system call	waiting
(f) after the blocking system call is complete	ready
(g) after it is interrupted by a hardware interrupt	ready

版本 C (2015) 答案

Situation	State
(a) after it is created (forked) by a (parent) process	ready
(b) before it is selected by a short-term scheduler	ready
(c) after it is selected by a short-term scheduler	running
(d) after it invokes a blocking system call	waiting
(e) after the blocking system call is complete	ready
(f) after it is interrupted by a hardware interrupt	ready
(g) after its time slice expired	ready

⚠ 高頻易錯：

- hardware interrupt 後 → **ready** (不是 waiting ! 行程沒在等任何 I/O 事件, 只是被搶占)
- time slice 到期後 → **ready** (只是被搶占, 可以立即再執行)
- blocking system call 之前 → **running** (還沒呼叫, 行程還在 CPU 上)
- blocking system call 之後 → **waiting** (等待 I/O 完成)

Q22 | Process State Diagram — Fill in Transitions (2016)

```

ready    → running    : (ii) be dispatched by a short-term scheduler
running  → ready      : (i)  be interrupted
running  → waiting    : (iii) perform I/O
waiting  → ready      : (iv) have I/O completed
running  → terminated: (v)  exit

```

Q23 | fork() Output Analysis

版本 A (2016)

(1) Output:

```
value = 10    (child process prints first: 5 + 5 = 10)
value = 20    (parent process prints after wait: 5 + 15 = 20)
```

Explanation: After `fork()`, parent and child have **independent copies** of `value` (both start at 5). Child adds 5 → prints 10. Parent calls `wait(NULL)` before printing, so parent's value += 15 → prints 20.

(2) Effect of removing `wait(NULL)`:

- Parent and child run **concurrently**; output order is **non-deterministic**.
- If parent finishes before child, the child becomes an **orphan process** (adopted by `init/systemd`).
- If parent exits without calling `wait()`, the child's PCB remains after it terminates → **zombie process**.

版本 B (2012)

Execution trace:

First `fork()` (`i = 0`):

- After fork: both parent and child execute `i = i + 5` → `i = 5`
- **Child** (`pid == 0`): `i = 5 + 5 = 10` → prints `i = 10`
- **Parent** (`pid > 0`): waits for child, then prints `i = 5`

Second `fork()` (from original parent, `i = 5`):

- After fork: both execute `i = i + 5` → `i = 10`
- **New child**: `i = 10 + 5 = 15` → prints `i = 15`
- **Original parent**: waits for child, then prints `i = 10`

Output (deterministic due to `wait()`):

```
i = 10
i = 5
i = 15
i = 10
```

Q24 | Shared Memory — Calculate $\Sigma n!$ **Complete program:**

```
#include <sys/shm.h>
#include <sys/stat.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
```



```

#include <sys/wait.h>

int main(int argc, char *argv[])
{
    int N = atoi(argv[1]);
    pid_t pid;
    int segment_id;
    int *shared_memory;

    // (A)(B) allocate and attach a shared memory segment
    segment_id = shmget(IPC_PRIVATE, sizeof(int), S_IRUSR | S_IWUSR);
    shared_memory = (int *) shmat(segment_id, NULL, 0);

    *shared_memory = 0; // (C) initialization

    for (int i = N; i > 0; i--) // (D) create child processes
    {
        pid = fork(); // (E)

        if (pid < 0) { // (F)
            fprintf(stderr, "Fork Failed");
            exit(-1);
        } else if (pid == 0) { // (G) child part
            // (H) do factorial calculation and save it
            int fact = 1;
            for (int j = i; j > 0; j--) fact *= j;
            *shared_memory += fact;
            // (I) detach shared memory and exit
            shmdt(shared_memory);
            exit(0);
        } else { // parent part
            wait(NULL); // (J) wait for child
        }
    }

    printf("the result is %d\n", *shared_memory);

    // (K)(L) detach and remove the shared memory segment
    shmdt(shared_memory);
    shmctl(segment_id, IPC_RMID, NULL);

    return 0;
}

```

Q25 | Shared Memory — Calculate $2x+3y-5$ ★★

Blank answers:

Blank	Answer	說明
(A)	shmget	建立共享記憶體區段

Blank	Answer	說明
(B)	<code>shmat</code>	附加共享記憶體到行程位址空間
(C)	<code>0</code>	初始化為 0
(D)	<code>fork</code>	建立子行程
(E)	<code>0</code>	<code>pid == 0</code> 表示子行程
(F)	<code>i == 2 → *shared_memory += 2 * x</code>	第一個子行程計算 2x
(G)	<code>i == 1 → *shared_memory += 3 * y</code>	第二個子行程計算 3y
(H)	<code>wait(NULL)</code>	父行程等待子行程完成，防止 race condition
(I)	<code>shmctl</code>	移除共享記憶體區段

Complete code (from 2015midA):

```

int main(int argc, char *argv[])
{
    int x = atoi(argv[1]);
    int y = atoi(argv[2]);
    int i;
    pid_t pid;
    const int sh_size = 4;

    int segment_id = shmget(IPC_PRIVATE, sh_size, S_IRUSR | S_IWUSR);
    int *shared_memory = (int *) shmat(segment_id, NULL, 0);
    *shared_memory = 0;

    for (i = 2; i > 0; i--) {
        pid = fork();
        if (pid < 0) {
            fprintf(stderr, "Fork Failed");
            exit(-1);
        } else if (pid == 0) {
            if (i == 2) *shared_memory += 2 * x;
            if (i == 1) *shared_memory += 3 * y;
            shmdt(shared_memory);
            exit(0);
        } else {
            wait();
        }
    }

    *shared_memory -= 5;
    printf("the result is %d\n", *shared_memory);
    shmdt(shared_memory);
    shmctl(segment_id, IPC_RMID, NULL);
    return 0;
}

```

Q26 | Shell Interface Program ★★

Blank answers:

Blank	Answer	說明
(A)	<code>cin >> inputBuffer</code> 或 <code>!feof(stdin)</code>	持續讀取輸入
(B)	讀取輸入並檢查是否以 <code>&</code> 結尾，設定 <code>background</code>	見下方邏輯
(C)	<code>fork + exec + wait</code> 邏輯	見下方完整程式

完整邏輯（來自 2016midA）：

```
int main(void)
{
    char inputBuffer[80];
    bool background;

    while (cin >> inputBuffer) // (A) 持續接受指令
    {
        cout << endl << " COMMAND->";
        pid_t pid;

        // step 1: check if command ends with '&'
        if (inputBuffer[strlen(inputBuffer)-1] == '&') {
            background = true;
            inputBuffer[strlen(inputBuffer)-1] = '\0'; // remove '&'
        } else {
            background = false;
        }

        // step 2: fork and execute
        pid = fork();

        if (pid == 0) {
            exec(inputBuffer); // child executes command
        } else if (pid > 0) {
            if (background == false)
                wait(NULL); // foreground: parent waits
            // background: parent continues immediately
        }

    } // end of while
} // end of main
```

Key points:

- `background = true` → child runs in background, parent does **not** call `wait()`
- `background = false` → parent calls `wait(NULL)` and waits for child to finish

肆、執行緒 (Multithreaded Programming)

Q27 | Why Process Creation is Costly vs Thread Creation

Every process contains program code (text), stack, heap, and more in memory. **Creating a process duplicates the entire address space** of the parent — including text, data, and heap — requiring significant memory allocation and copying.

Creating a thread only requires allocating a new **thread ID, program counter, register set, and stack**. Threads **share** the code, data, heap, and open files of the parent process. In Solaris, thread creation is approximately **30 times faster** than process creation.

Q28 | Many-to-One Thread Model — Pros and Cons

The many-to-one model maps **multiple user threads to one kernel thread**.

-  **Pros:** Thread management is done entirely in **user space**, so it is efficient and requires no system calls for thread operations. Quick in executing because it will not invoke system calls.
 -  **Cons:** If any user thread makes a **blocking system call**, the **entire process** (all threads) is blocked. Threads **cannot run in parallel** on multiprocessors since there is only one kernel thread.
-

Q29 | One-to-One Thread Model — Pros and Cons

The one-to-one model maps **each user thread to one kernel thread**.

-  **Pros:** One thread blocking does **not affect** others. Threads can **truly run in parallel** on multiprocessors, supporting real concurrency.
 -  **Cons:** Creating a user thread requires creating a kernel thread → **higher overhead**. The number of kernel threads may be limited, restricting scalability. **Linux and Windows use this model**.
-

Q30 | Compare Many-to-One and One-to-One

The many-to-one model has a kernel thread that allocates many user threads. It can be quick in executing because it will not invoke system calls. **The disadvantage** is that if one user thread is blocked, all the threads will be blocked, too.

The one-to-one model can fix that blocking problem but **is limited in number** — there can't be too many threads and it might be unable to efficiently run a program.

Q31 | Scheduler Activation — Problem it Resolves

In the **many-to-many model**, when a user thread performs a blocking system call (e.g., I/O), the kernel blocks the **underlying kernel thread**. Since multiple user threads share kernel threads, the kernel has **no way to notify the thread library** about this blocking. As a result, other user threads that are ready to run **cannot execute** — the CPU is idle even though there are runnable threads.

Q32 | Scheduler Activation — How it Works ★

Scheduler activation uses **LWP (Lightweight Process)** as virtual processors between the kernel and the thread library.

- 1. A user thread is about to make a **blocking system call**.
- 2. The kernel detects this and sends an **upcall** (upward notification) to the **thread library**.
- 3. The **upcall handler** in the thread library:
 - Saves the state of the blocking thread
 - Uses the **scheduler** to select another ready user thread
 - Schedules that thread on an available LWP to execute
- 4. When the blocking I/O completes, the kernel sends another **upcall** to the thread library.
- 5. The **upcall handler** marks the original thread as **ready** and places it back in the ready queue.

伍、CPU 排程 (CPU Scheduling)

Q33 | Preemptive Scheduling — Four Situations

The four circumstances under which CPU scheduling decisions take place:

#	Situation	Type
①	Process switches from running → waiting (e.g., I/O request)	Nonpreemptive
②	Process switches from running → ready (e.g., interrupt occurs)	Preemptive ★
③	Process switches from waiting → ready (e.g., I/O complete)	Preemptive ★
④	Process terminates	Nonpreemptive

Nonpreemptive: Scheduling only occurs when a process voluntarily gives up the CPU (cases ① and ④).

Preemptive: The OS can forcibly take away the CPU (cases ② and ③).

Q34 | Priority Scheduling — Starvation Solution

When there are infinitely many processes with high priority, processes with lower priority may **never be executed** — this is called **starvation**.

The solution is **aging** (老化) :

The system maintains a value that increases with time. If a process has been waiting long enough ("old enough"), its **priority gradually increases** until it is high enough to be selected for execution. This ensures that even if high-priority processes keep arriving, low-priority processes will eventually execute.

Q35 | Multilevel Feedback Queue — Gantt Chart ★★

Time	Process	Queue	Event
0–4	P ₁	Q2 (RR-4)	P ₁ uses 4ms quantum → demoted to Q1 (4ms remaining)
4–6	P ₂	Q2 (RR-4)	P ₂ arrives at t=2, enters Q2. Uses 2ms → first burst done, goes to I/O (2ms, finishes t=8)
6–8	P ₁	Q1 (RR-6)	P ₁ runs from Q1, uses 2ms
8	—	—	P ₂ returns from I/O → enters Q2 (higher priority) → preempts P₁ in Q1
8–12	P ₂	Q2 (RR-4)	P ₂ uses 4ms quantum → demoted to Q1 (12ms remaining)
12–14	P ₁	Q1 (RR-6)	P ₁ resumes, uses 2ms → first burst complete, goes to I/O (4ms, finishes t=18)
14–18	P ₂	Q1 (RR-6)	P ₂ runs, uses 4ms (2ms of Q1 quantum remaining)
18	—	—	P ₁ returns from I/O → enters Q2 (higher priority) → preempts P₂ in Q1
18–22	P ₁	Q2 (RR-4)	P ₁ uses 4ms quantum → demoted to Q1 (4ms remaining)
22–24	P ₂	Q1 (RR-6)	P ₂ uses 2ms (remaining Q1 quantum) → demoted to Q0 (6ms remaining)
24–28	P ₁	Q1 (RR-6)	P ₁ uses 4ms → second burst complete. P₁ done at t=28
28–34	P ₂	Q0 (FCFS)	P ₂ uses 6ms → P₂ done at t=34

Time: 0 2 4 6 8 10 12 14 16 18 20 22 24 26
28 30 32 34

| | | | | | | | | | | | | | | |

| | | |

Q2: [—P₁—] [—P₂—] [—P₂—]

[—P₁—]

Q1: [—P₁—] [—P₁—] [—P₁—] [—P₂—]

[—P₁—]

Q0: [—P₂—]

簡化表：

0- 4: P₁ (Q2)

4- 6: P₂ (Q2)

```

6- 8: P1 (Q1)
8-12: P2 (Q2)
12-14: P1 (Q1)
14-18: P2 (Q1)
18-22: P1 (Q2)
22-24: P2 (Q1)
24-28: P1 (Q1)
28-34: P2 (Q0)

```

(b) 平均等待時間：

公式：Waiting Time = Finish - Arrival - Total CPU Burst - I/O Burst

Process	Arrival	Finish	CPU Burst	I/O	Waiting Time
P ₁	0	28	8+8 = 16ms	4ms	28-0-16-4 = 8 ms
P ₂	2	34	2+16 = 18ms	2ms	34-2-18-2 = 12 ms

Average Waiting Time = (8 + 12) / 2 = 10 ms

⚠ 注意：I/O 等待時間不計入 Waiting Time。

附錄：常見錯誤

題目	錯誤答案	正確答案	原因
hardware interrupt 後的狀態	waiting	ready	中斷只搶占 CPU，行程不在等任何事件
time slice 到期後的狀態	waiting	ready	只是被搶占，可以立即再執行
blocking system call 之前的狀態	waiting	running	還沒呼叫，行程還在 CPU 上
fork() 子行程的回傳值	1	0	父行程才回傳子 PID，子行程回傳 0
分離共享記憶體函數	shmctl	shmdt	shmdt = detach ; shmctl(IPC_RMID) = 刪除
Linux 使用的執行緒模型	Many-to-Many	One-to-One	Linux 每個 user thread 對應一個 kernel thread
mode bit = 0 代表	user mode	kernel mode	0 = kernel, 1 = user
Time-sharing 與 Multiprogramming 的關係	相同	Time-sharing 是 Multiprogramming 的延伸	

附錄：各年題目索引

題目	2011	2012	2013	2014	2015	2016
Q1 Interrupt-based vs Busy-waiting	Q1a	—	Q1a	Q1a	—	—
Q2 Hardware Interrupt Handling	Q1b	Q1b	Q1b	Q1b	—	—
Q3 Modern Interrupt Mechanism	—	—	—	—	—	Q1a
Q4 I/O Cycle Flowchart	—	—	—	—	Q1b	Q1b
Q5 Interrupt Vector	Q1c	—	—	Q1c	—	—
Q6 Device Controller vs Driver	—	—	Q2	—	Q1a	—
Q7 Concurrent I/O and CPU	—	Q1a	—	—	—	—
Q8 API → System Call	Q3	Q2a	Q4a	Q4a	Q4a	Q2a
Q9 Why Prefer API	Q3	Q2b	Q4b	Q4b	Q4b	Q2b
Q10 Passing Many Parameters	—	—	Q4c	Q4c	—	—
Q11 Layered OS Pros/Cons	Q4	—	—	—	—	Q2c
Q12 Microkernel Pros/Cons	—	Q3	—	—	—	—
Q13 Modular Kernel Advantages	—	—	Q5	—	—	—
Q14 Dual-mode	—	Q1d	Q3	—	—	—
Q15 Higher-level Language	—	—	—	—	Q2	—
Q16 CLI Two Approaches	—	—	—	—	Q3	—
Q17 Multiprogramming vs Timesharing	Q2	Q1c	—	—	—	Q1c
Q18 Asymmetric vs Symmetric	—	—	—	Q2	—	—
Q19 OS Supports New Devices	—	—	—	Q3	—	Q1d
Q20 4 Components of Process	—	—	—	Q5	—	—
Q21 Process State Situations	Q5	Q4	Q6	—	Q5	—
Q22 Process State Diagram	—	—	—	—	—	Q3a
Q23 fork() Output	—	Q5	—	—	—	Q3b
Q24 Shared Memory $\Sigma n!$	Q6	—	—	Q9	—	—
Q25 Shared Memory $2x+3y-5$	—	Q6	—	—	Q9	—
Q26 Shell Program	—	—	Q8	—	—	Q5
Q27 Thread vs Process Cost	—	—	—	—	Q6a	Q4a
Q28 Many-to-One Pros/Cons	—	Q7a	Q7a	—	—	—

題目	2011	2012	2013	2014	2015	2016
Q29 One-to-One Pros/Cons	—	—	Q7b	—	—	Q4b
Q30 Compare M:1 vs 1:1	—	—	—	—	Q6b	—
Q31 Scheduler Activation Problem	Q7a	—	—	Q6a	—	—
Q32 Scheduler Activation Operation	Q7b	Q7b	—	Q6b	—	—
Q33 Preemptive 4 Situations	—	—	—	Q7	—	—
Q34 Priority Starvation/Aging	—	—	—	—	Q7	—
Q35 Multilevel Feedback Queue	—	—	—	Q8	Q8	—

End of 考古題解答。